Risk-Based Model Selection for Fixed and Adaptive Nonparametric Smoothers

Nahyun Lee

06/12/2025

Contents

Abstract . . . 1
1. Introduction . . . 1
2. Data and Computational Setup . . . 2
3. Fixed-Bin Smoothers on the Motorcycle Crash Data . . . 5
4. Adaptive Regressogram on the Motorcycle Crash Data . . . 16
5. Histogram Risk Estimation on `qer.csv` . . . 19
6. Conclusion . . . 21

Abstract

This project investigates nonparametric smoothing and model selection. Using the motorcycle crash dataset, we compare fixed-bin regressograms, LOWESS smoothing, and an adaptive regressogram built by greedy recursive partitioning based on RSS reduction. This study focuses on explicit **risk estimation** as a principled criterion for model selection, rather than relying on visual inspection alone. We conclude with a complementary histogram-selection exercise on `qer.csv`, showing that the same penalized-risk logic extends beyond regression.

1. Introduction

The motorcycle crash dataset records head acceleration over time during a simulated crash event. Because the underlying signal is nonlinear and highly variable, nonparametric methods are natural tools for recovering its structure.

This report studies two related questions:

1. how to choose the tuning parameters of standard smoothers such as regressograms and LOWESS,
2. how to extend the regressogram idea to an adaptive, data-driven partition of the input space.

The common framework is **risk-based model selection**. For fixed-bin smoothers, we compare covariance-penalty ideas, bootstrap optimism, and cross-validation. For adaptive binning, we use a stronger complexity penalty to keep recursive partitioning from overfitting.

This part of the report is especially influenced by Bradley Efron's paper, *The Estimation of Prediction Error: Covariance Penalties and Cross-Validation*. The covariance-penalty calculations, the bootstrap-optimism interpretation, and the Figure 6-like diagnostic are all motivated by that paper's general framework for estimating prediction error.

2. Data and Computational Setup

2.1 Data Sources

```
locate_data_file <- function(filename) {
  candidates <- c(
    file.path(getwd(), filename),
    file.path(getwd(), "data", filename),
    file.path(path.expand("~/Desktop"), filename)
  )

  existing <- candidates[file.exists(candidates)]
  if (length(existing) == 0) {
    stop(paste("Could not find", filename, "in the working directory, ./data, or ~/Desktop."))
  }
  existing[1]
}

mct_path <- locate_data_file("mct.csv")
qer_path <- locate_data_file("qer.csv")

crash_data <- read.csv(mct_path)
qer_data <- read.csv(qer_path)

x <- crash_data$xx
y <- crash_data$yy
n <- length(y)

x_qer <- qer_data$xx
n_qer <- length(x_qer)

cat("Sample size for motorcycle data:", n, "\n")
```

```
## Sample size for motorcycle data: 973
```

```
cat("Sample size for qer data:", n_qer, "\n")
```

```
## Sample size for qer data: 1024
```

2.2 Helper Functions

```
calc_rss <- function(y_vals) {
  if (length(y_vals) == 0) return(0)
  mean_y <- mean(y_vals)
```

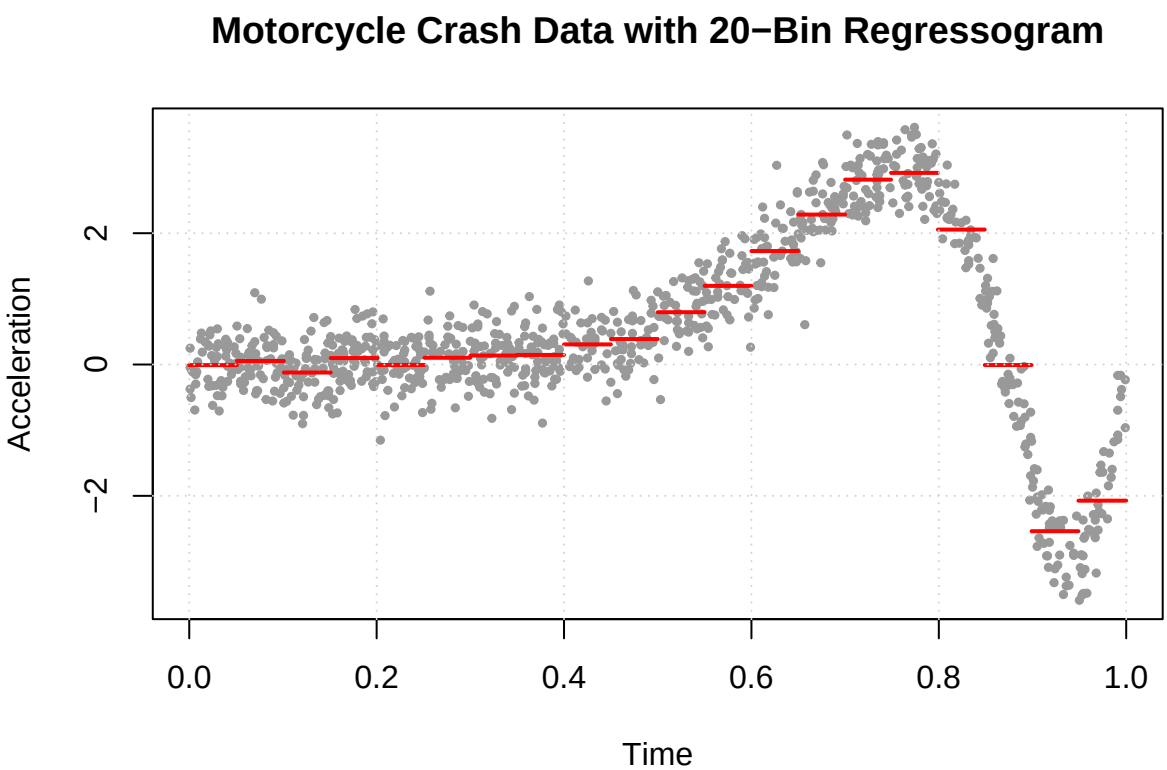
```r
  sum((y_vals - mean_y)^2)
}

fit_and_predict_regressogram <- function(x_train, y_train, k, x_new) {
  full_range <- range(c(x_train, x_new), na.rm = TRUE)
  if (diff(full_range) == 0) {
    return(rep(mean(y_train, na.rm = TRUE), length(x_new)))
  }

  buffer <- 1e-6 * diff(full_range)
  breaks <- seq(full_range[1] - buffer, full_range[2] + buffer, length.out = k + 1)

  train_bins <- cut(x_train, breaks = breaks, include.lowest = TRUE)
  mean_y_by_bin <- tapply(y_train, train_bins, mean, na.rm = TRUE)
  new_bins <- cut(x_new, breaks = breaks, include.lowest = TRUE)

  preds <- mean_y_by_bin[match(new_bins, names(mean_y_by_bin))]
  preds[is.na(preds)] <- mean(y_train, na.rm = TRUE)
  preds
}

predict_regressogram <- function(x_train, y_train, x_new, k) {
  fit_and_predict_regressogram(x_train, y_train, k, x_new)
}

fit_adaptive_regressogram <- function(x_all, y_all, min_split = 5, max_leaf = 5) {
  full_data <- data.frame(xx = x_all, yy = y_all)
  active_bins <- list(full_data)

  split_records <- data.frame(
    bins_k = integer(),
    split_location = numeric(),
    rss_after_split = numeric()
  )

  while (TRUE) {
    best_split <- list(
      rss_k_plus_1 = Inf,
      parent_bin_idx = NA,
      split_value = NA,
      left_child = NULL,
      right_child = NULL
    )

    can_split <- FALSE

    for (i in seq_along(active_bins)) {
      parent_data <- active_bins[[i]]
      if (nrow(parent_data) < min_split) next

      unique_x <- sort(unique(parent_data$xx))
      if (length(unique_x) < 2) next
      can_split <- TRUE
```

```r
      rss_other_bins <- 0
      for (j in seq_along(active_bins)) {
        if (j != i) rss_other_bins <- rss_other_bins + calc_rss(active_bins[[j]]$yy)
      }

      mids <- (unique_x[-length(unique_x)] + unique_x[-1]) / 2

      for (sp_value in mids) {
        left_data <- parent_data[parent_data$xx < sp_value, ]
        right_data <- parent_data[parent_data$xx > sp_value, ]
        if (nrow(left_data) == 0 || nrow(right_data) == 0) next

        total_rss <- rss_other_bins + calc_rss(left_data$yy) + calc_rss(right_data$yy)

        if (total_rss < best_split$rss_k_plus_1) {
          best_split$rss_k_plus_1 <- total_rss
          best_split$parent_bin_idx <- i
          best_split$split_value <- sp_value
          best_split$left_child <- left_data
          best_split$right_child <- right_data
        }
      }
    }

    if (!can_split || is.infinite(best_split$rss_k_plus_1)) break

    split_records <- rbind(split_records, data.frame(
      bins_k = length(active_bins) + 1,
      split_location = best_split$split_value,
      rss_after_split = best_split$rss_k_plus_1
    ))

    parent_idx <- best_split$parent_bin_idx
    new_active_bins <- list()
    for (j in seq_along(active_bins)) {
      if (j != parent_idx) new_active_bins[[length(new_active_bins) + 1]] <- active_bins[[j]]
    }
    new_active_bins[[length(new_active_bins) + 1]] <- best_split$left_child
    new_active_bins[[length(new_active_bins) + 1]] <- best_split$right_child
    active_bins <- new_active_bins

    max_in_bin <- max(sapply(active_bins, nrow))
    if (max_in_bin <= max_leaf) break
  }

  split_records
}
```

3. Fixed-Bin Smoothers on the Motorcycle Crash Data

3.1 A 20-Bin Regressogram

```
bins_20 <- cut(x, breaks = 20, include.lowest = TRUE)
mean_y_20 <- tapply(y, bins_20, mean, na.rm = TRUE)

bin_levels_20 <- levels(bins_20)
lower_bounds_20 <- as.numeric(sub("\\(?\\[?(-?[0-9.e+-]+),.*", "\\1", bin_levels_20))
upper_bounds_20 <- as.numeric(sub("[^,]*,(-?[0-9.e+-]+)\\]?", "\\1", bin_levels_20))

regressogram_df_20 <- data.frame(
  x_start = lower_bounds_20,
  x_end = upper_bounds_20,
  y_mean = mean_y_20[bin_levels_20]
)
regressogram_df_20 <- regressogram_df_20[!is.na(regressogram_df_20$y_mean), ]

plot(
  x, y,
  main = "Motorcycle Crash Data with 20-Bin Regressogram",
  xlab = "Time",
  ylab = "Acceleration",
  pch = 19,
  cex = 0.5,
  col = "grey60"
)
with(regressogram_df_20, segments(x_start, y_mean, x_end, y_mean, col = "red", lwd = 2))
grid()
```



This plot gives a baseline piecewise-constant approximation to the signal. The central question is how many bins should be used when the goal is prediction rather than visual roughness alone.

3.2 Residual Variance Estimate

```
k_sigma_est <- floor(n / log(n))
if (k_sigma_est < 1) k_sigma_est <- 1

bins_k_sigma <- cut(x, breaks = k_sigma_est, include.lowest = TRUE)
lm_fit_k_sigma <- lm(y ~ factor(bins_k_sigma) - 1)
sigma_hat <- summary(lm_fit_k_sigma)$sigma
sigma_hat_sq <- sigma_hat^2

cat("Residual standard deviation estimate:", round(sigma_hat, 4), "\n")
```

```
## Residual standard deviation estimate: 0.3695
```

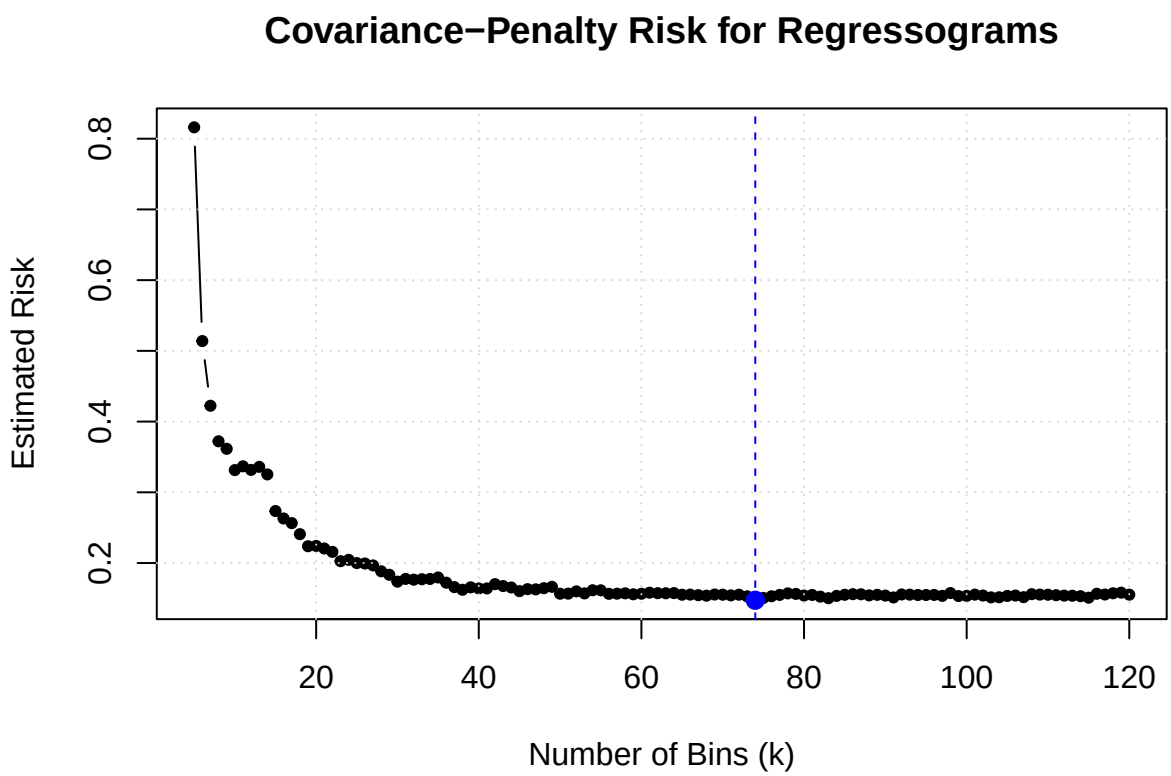
```
cat("Number of bins used for sigma estimation:", k_sigma_est, "\n")
```

```
## Number of bins used for sigma estimation: 141
```

The covariance-penalty calculation depends on the residual variance estimate `hat(sigma)^2`, so it is reasonable to ask whether this introduces circular reasoning. Here the estimate is taken from a relatively flexible pilot regressogram with `k = floor(n / log n)`, which is large enough to absorb broad signal structure without becoming so fine that variance is dominated by empty or nearly empty bins. In that sense, `hat(sigma)^2` is used as a calibration quantity from a sufficiently flexible preliminary fit, rather than being tuned to force a particular final value of `k`.

3.3 Covariance-Penalty Risk for Regressograms

```r
k_values <- 5:120
k_values <- k_values[k_values < n]
estimated_risks <- numeric(length(k_values))

for (i in seq_along(k_values)) {
  k <- k_values[i]
  current_bins <- cut(x, breaks = k, include.lowest = TRUE)
  lm_fit_current_k <- lm(y ~ factor(current_bins) - 1)
  rss_k <- deviance(lm_fit_current_k)
  estimated_risks[i] <- (rss_k / n) + (2 * sigma_hat_sq * k / n)
}

idx_cp <- which.min(estimated_risks)
k_cp <- k_values[idx_cp]
risk_cp_min <- estimated_risks[idx_cp]
rss_fixed_cp <- deviance(lm(y ~ factor(cut(x, breaks = k_cp, include.lowest = TRUE)) - 1))

plot(
  k_values, estimated_risks,
  type = "b",
  pch = 19,
  cex = 0.7,
  xlab = "Number of Bins (k)",
  ylab = "Estimated Risk",
  main = "Covariance-Penalty Risk for Regressograms"
)
points(k_cp, risk_cp_min, col = "blue", pch = 19, cex = 1.2)
abline(v = k_cp, col = "blue", lty = 2)
grid()
```




```r
cat("Optimal k by covariance penalty:", k_cp, "\n")
```

```
## Optimal k by covariance penalty: 74
```

```r
cat("RSS at optimal fixed-bin k:", round(rss_fixed_cp, 4), "\n")
```

```
## RSS at optimal fixed-bin k: 123.1776
```

3.4 Bootstrap Optimism

```r
B_bootstrap <- 200
k_seq_boot <- 5:120
k_seq_boot <- k_seq_boot[k_seq_boot < n]

set.seed(123)
risk_boot <- numeric(length(k_seq_boot))

for (i in seq_along(k_seq_boot)) {
  k <- k_seq_boot[i]
  preds_original <- fit_and_predict_regressogram(x, y, k, x)
  apparent_error <- mean((y - preds_original)^2, na.rm = TRUE)

  optimism_sum <- 0
  for (b in 1:B_bootstrap) {
    boot_idx <- sample(1:n, n, replace = TRUE)
    x_boot <- x[boot_idx]
```

```r
    y_boot <- y[boot_idx]

    preds_boot_boot <- fit_and_predict_regressogram(x_boot, y_boot, k, x_boot)
    err_boot_boot <- mean((y_boot - preds_boot_boot)^2, na.rm = TRUE)

    preds_boot_orig <- fit_and_predict_regressogram(x_boot, y_boot, k, x)
    err_boot_orig <- mean((y - preds_boot_orig)^2, na.rm = TRUE)

    optimism_sum <- optimism_sum + (err_boot_orig - err_boot_boot)
  }

  risk_boot[i] <- apparent_error + optimism_sum / B_bootstrap
}

idx_boot <- which.min(risk_boot)
k_boot <- k_seq_boot[idx_boot]
bootstrap_risk_min <- risk_boot[idx_boot]
```

Bootstrap optimism provides a resampling-based estimate of prediction risk. On the current computation, it selects `k = 74` with minimum bootstrap-adjusted risk 0.14591. Instead of presenting another risk curve similar to the covariance-penalty plot, the report uses the Figure-6-style diagnostic to illustrate how prediction optimism varies across the design space.

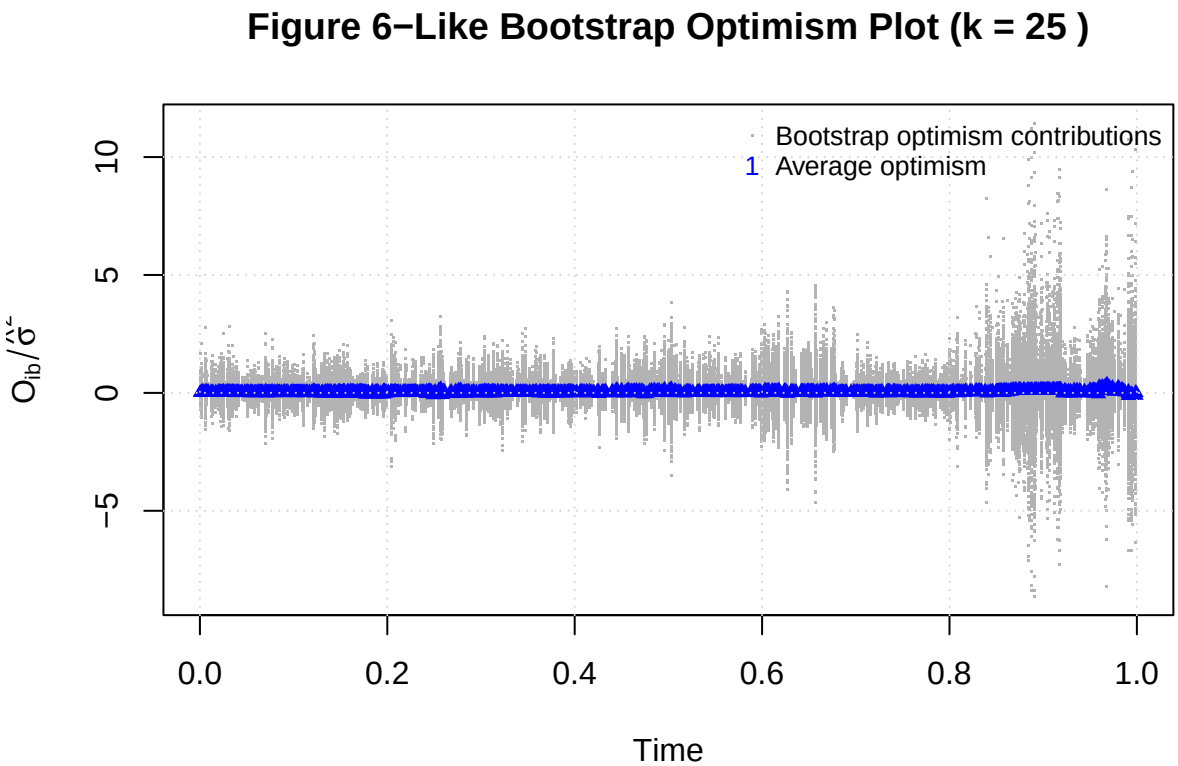
3.5 Figure 6-Like Diagnostic for Bootstrap Optimism

The next plot is a **Figure 6-like diagnostic**, modeled on the optimism plot used by **Bradley Efron** in *The Estimation of Prediction Error: Covariance Penalties and Cross-Validation* to visualize how prediction optimism varies across the design space. Here we fix `k = 25` and examine pointwise bootstrap optimism contributions, scaled by the residual variance estimate.

This figure is not the main model-selection device by itself. Instead, it gives a visual explanation for why bootstrap-based risk estimation can support a model choice that differs from simple covariance-penalty calculations: if optimism is concentrated only in certain unstable regions, then additional complexity may still be acceptable outside those regions.

```r
k_fig6 <- 25

preds_k_fig6_orig <- fit_and_predict_regressogram(x, y, k_fig6, x)
err_sq_orig <- (y - preds_k_fig6_orig)^2
opt_contrib_mat <- matrix(NA_real_, nrow = n, ncol = B_bootstrap)

for (b_idx in 1:B_bootstrap) {
  boot_indices <- sample(1:n, n, replace = TRUE)
  x_boot <- x[boot_indices]
  y_boot <- y[boot_indices]

  preds_boot_on_orig <- fit_and_predict_regressogram(x_boot, y_boot, k_fig6, x)
  err_sq_boot_on_orig <- (y - preds_boot_on_orig)^2

  opt_contrib_mat[, b_idx] <- err_sq_boot_on_orig - err_sq_orig
}

opt_contrib_scaled <- opt_contrib_mat / sigma_hat_sq
```

```r
avg_opt_scaled <- rowMeans(opt_contrib_scaled, na.rm = TRUE)

plot_df_dots <- data.frame(
  x_val = rep(x, B_bootstrap),
  optimism_scaled = as.vector(opt_contrib_scaled)
)
plot_df_dots <- na.omit(plot_df_dots)

avg_df <- data.frame(
  x_val = x,
  optimism_avg = avg_opt_scaled
)
avg_df <- na.omit(avg_df)

plot(
  plot_df_dots$x_val,
  plot_df_dots$optimism_scaled,
  pch = ".",
  col = "grey70",
  xlab = "Time",
  ylab = expression(O[i*b] / hat(sigma)^2),
  main = paste("Figure 6-Like Bootstrap Optimism Plot (k =", k_fig6, ")")
)
points(avg_df$x_val, avg_df$optimism_avg, pch = 17, col = "blue", cex = 0.8)
grid()
legend(
  "topright",
  legend = c("Bootstrap optimism contributions", "Average optimism"),
  col = c("grey70", "blue"),
  pch = c(".", 17),
  bty = "n",
  cex = 0.8
)
```

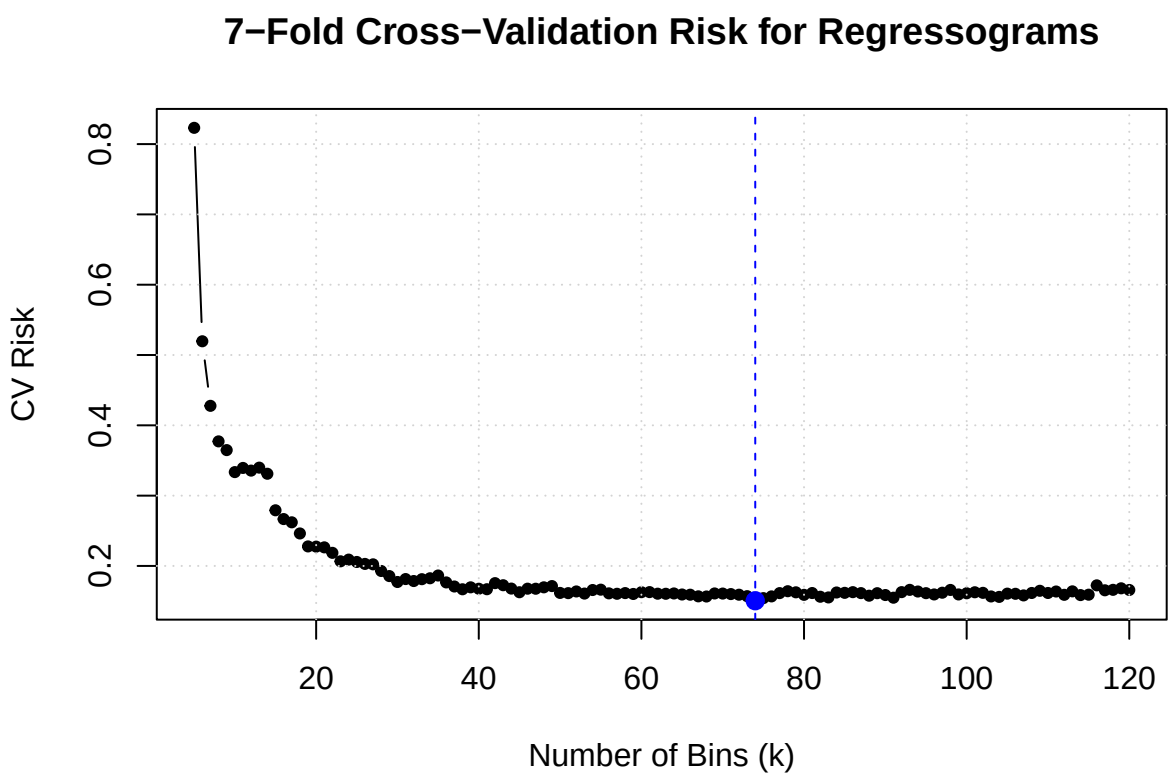


Following the diagnostic framework presented in Efron (2004), this plot shows that optimism is not uniformly distributed across the input space. Regions with greater instability or sparser local support exhibit larger optimism contributions, while more stable regions show relatively small optimism. In particular, the increase in optimism toward the later time range is plausibly driven by both effects at once: the crash signal becomes locally more volatile there, and the design is also relatively sparse near the edge of the observed time domain. That pattern helps explain why bootstrap optimism may favor a larger `k` than covariance-penalty or cross-validation methods: additional local flexibility is not equally risky everywhere.

3.6 7-Fold Cross-Validation

```
k_folds <- 7
set.seed(123)
folds <- sample(rep(1:k_folds, length.out = n))

k_values_cv <- 5:120
k_values_cv <- k_values_cv[k_values_cv < n]
estimated_risks_cv <- numeric(length(k_values_cv))

for (i in seq_along(k_values_cv)) {
  k <- k_values_cv[i]
  mse_folds <- numeric(k_folds)

  for (j in 1:k_folds) {
    test_indices <- which(folds == j)
    train_indices <- which(folds != j)

    x_train <- x[train_indices]
    y_train <- y[train_indices]
```

```r
    x_test <- x[test_indices]
    y_test <- y[test_indices]

    y_pred <- predict_regressogram(x_train, y_train, x_test, k)
    mse_folds[j] <- mean((y_test - y_pred)^2, na.rm = TRUE)
  }

  estimated_risks_cv[i] <- mean(mse_folds, na.rm = TRUE)
}

idx_cv <- which.min(estimated_risks_cv)
k_cv <- k_values_cv[idx_cv]

plot(
  k_values_cv, estimated_risks_cv,
  type = "b",
  pch = 19,
  cex = 0.7,
  xlab = "Number of Bins (k)",
  ylab = "CV Risk",
  main = "7-Fold Cross-Validation Risk for Regressograms"
)
points(k_cv, estimated_risks_cv[idx_cv], col = "blue", pch = 19, cex = 1.2)
abline(v = k_cv, col = "blue", lty = 2)
grid()
```

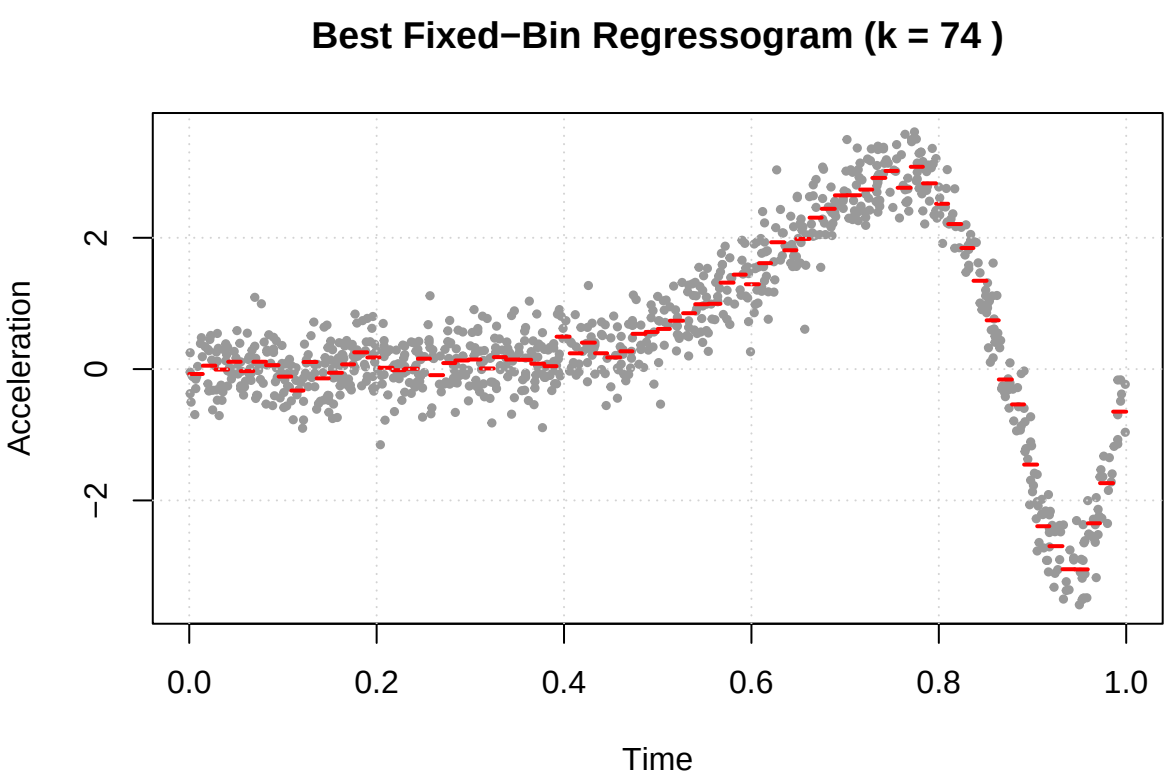



```r
cat("Optimal k by 7-fold CV:", k_cv, "\n")
```

```
## Optimal k by 7-fold CV: 74
```

3.7 Best Fixed-Bin Regressogram

```
bins_best <- cut(x, breaks = k_cp, include.lowest = TRUE)
mean_y_best <- tapply(y, bins_best, mean, na.rm = TRUE)

bin_levels_best <- levels(bins_best)
lower_bounds_best <- as.numeric(sub("\\(?\\[?(-?[0-9.e+-]+),.*", "\\1", bin_levels_best))
upper_bounds_best <- as.numeric(sub("[^,]*,(-?[0-9.e+-]+)\\]?", "\\1", bin_levels_best))

regressogram_df_best <- data.frame(
  x_start = lower_bounds_best,
  x_end = upper_bounds_best,
  y_mean = mean_y_best[bin_levels_best]
)
regressogram_df_best <- regressogram_df_best[!is.na(regressogram_df_best$y_mean), ]

plot(
  x, y,
  main = paste("Best Fixed-Bin Regressogram (k =", k_cp, ")"),
  xlab = "Time",
  ylab = "Acceleration",
  pch = 19,
  cex = 0.5,
  col = "grey60"
)
with(regressogram_df_best, segments(x_start, y_mean, x_end, y_mean, col = "red", lwd = 2))
grid()
```

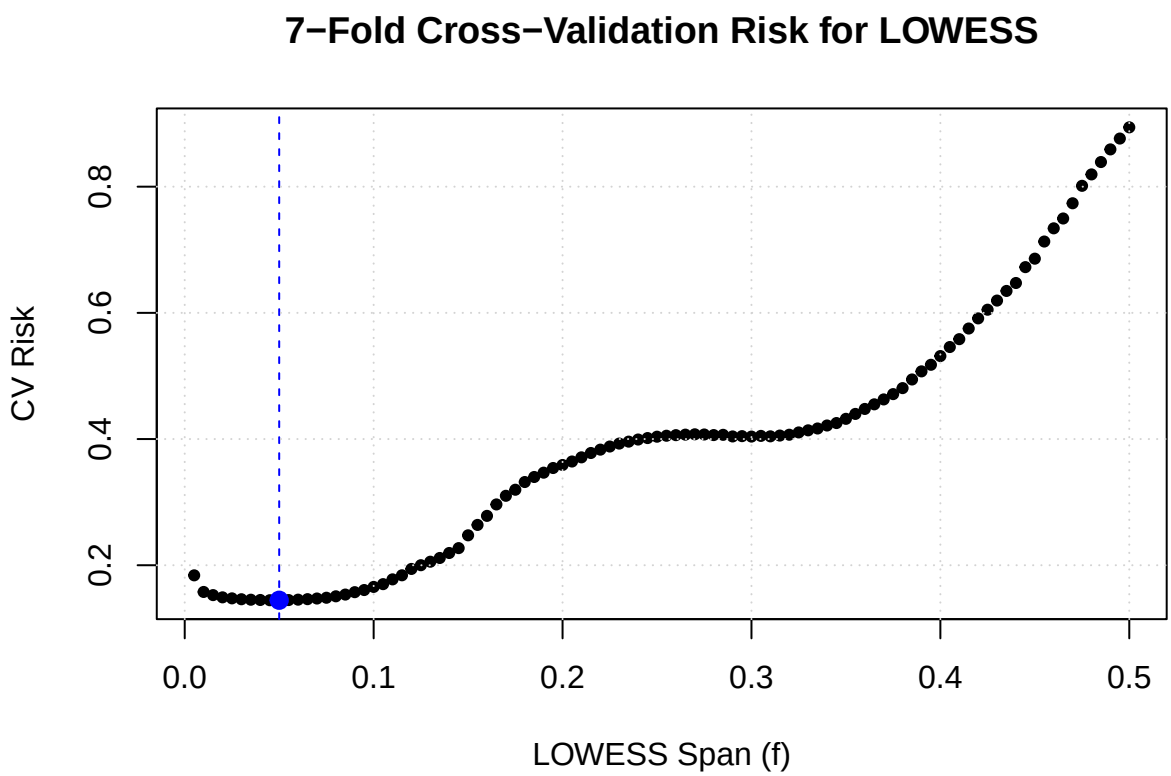


The fixed-bin analysis reproduces the main result obtained earlier: both the covariance-penalty criterion

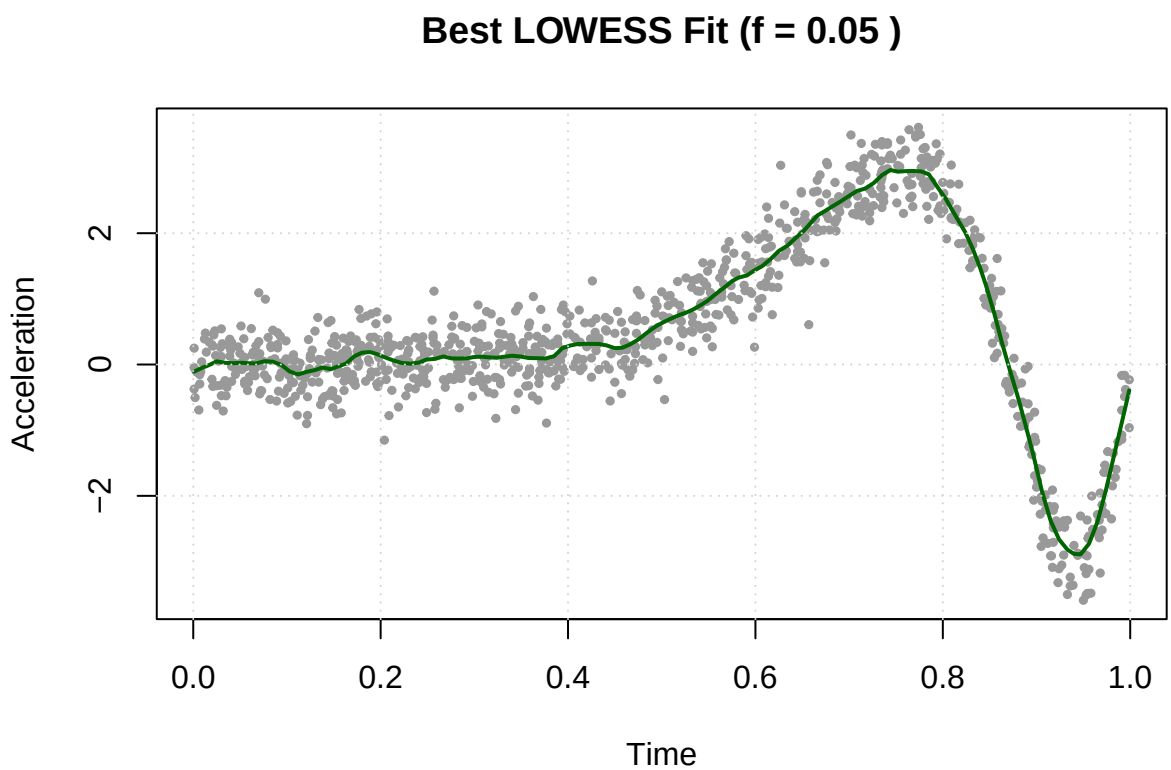
and cross-validation select `k = 74`. covariance penalty and 7-fold cross-validation both prefer `k = 74`, while bootstrap optimism selects `k = 74` on the current computation. Relative to the penalty and cross-validation criteria, the bootstrap criterion can be more tolerant of extra local flexibility when optimism is concentrated in a limited portion of the design space rather than spread uniformly across all time points. Because bootstrap optimism is based on resampling, its selected value can vary somewhat across runs unless the random seed is fixed; in this report we use `set.seed(123)` to make the result reproducible.

3.8 LOWESS Model Selection

```r
f_values <- seq(0.005, 0.5, by = 0.005)
estimated_risks_lowess <- numeric(length(f_values))

for (i in seq_along(f_values)) {
  f <- f_values[i]
  mse_folds <- numeric(k_folds)

  for (j in 1:k_folds) {
    test_indices <- which(folds == j)
    train_indices <- which(folds != j)

    x_train <- x[train_indices]
    y_train <- y[train_indices]
    x_test <- x[test_indices]
    y_test <- y[test_indices]

    ord_train <- order(x_train)
    lowess_fit <- lowess(x_train[ord_train], y_train[ord_train], f = f)
    y_pred <- approx(lowess_fit$x, lowess_fit$y, xout = x_test, rule = 2)$y
    mse_folds[j] <- mean((y_test - y_pred)^2, na.rm = TRUE)
  }

  estimated_risks_lowess[i] <- mean(mse_folds, na.rm = TRUE)
}

idx_lowess <- which.min(estimated_risks_lowess)
f_lowess <- f_values[idx_lowess]

plot(
  f_values, estimated_risks_lowess,
  type = "b",
  pch = 19,
  cex = 0.7,
  xlab = "LOWESS Span (f)",
  ylab = "CV Risk",
  main = "7-Fold Cross-Validation Risk for LOWESS"
)
points(f_lowess, estimated_risks_lowess[idx_lowess], col = "blue", pch = 19, cex = 1.2)
abline(v = f_lowess, col = "blue", lty = 2)
grid()
```




```r
ord_all <- order(x)
lowess_best <- lowess(x[ord_all], y[ord_all], f = f_lowess)

plot(
  x, y,
  main = paste("Best LOWESS Fit (f =", round(f_lowess, 3), ")"),
  xlab = "Time",
  ylab = "Acceleration",
  pch = 19,
  cex = 0.5,
  col = "grey60"
)
lines(lowess_best$x, lowess_best$y, col = "darkgreen", lwd = 2)
grid()
```




```r
cat("Optimal LOWESS span:", round(f_lowess, 3), "\n")
```

```
## Optimal LOWESS span: 0.05
```

4. Adaptive Regressogram on the Motorcycle Crash Data

4.1 Recursive Partitioning Output

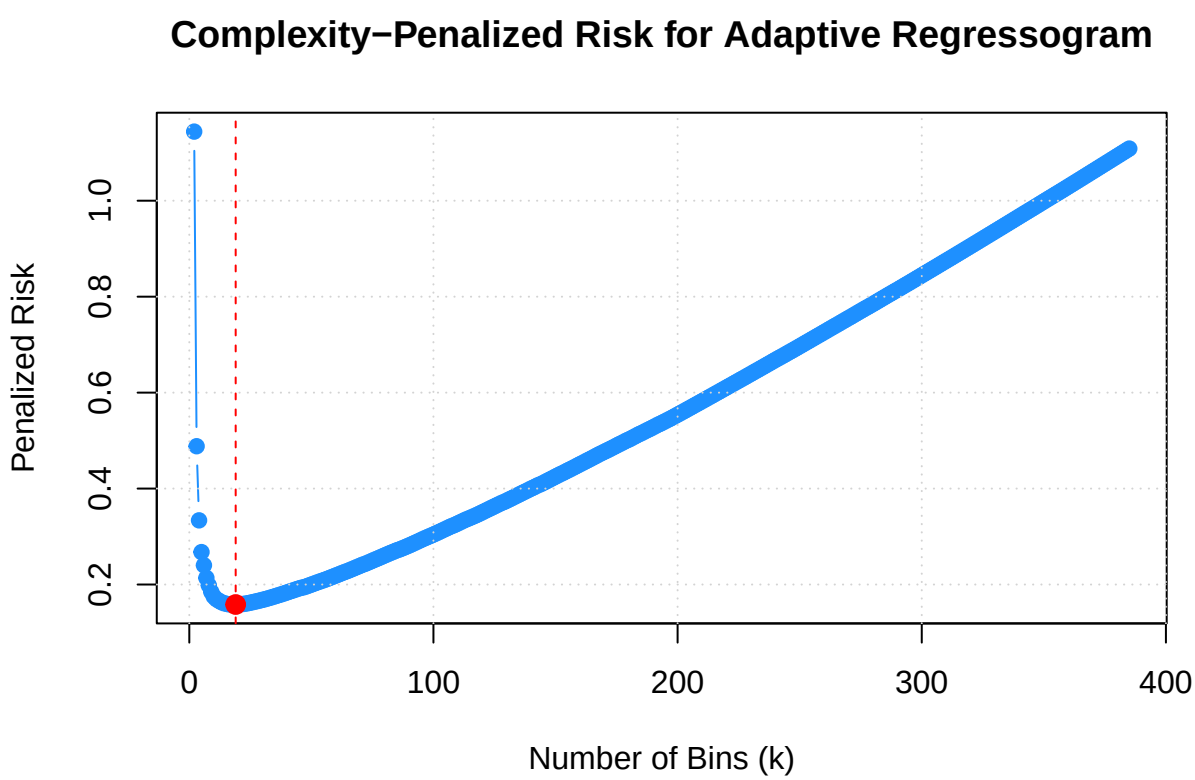
```r
split_results <- fit_adaptive_regressogram(x, y, min_split = 5, max_leaf = 5)
head(split_results)
```

```
##   bins_k split_location rss_after_split
## 1      2         0.8915       1112.5196
## 2      3         0.5615        473.5953
## 3      4         0.8430        322.2475
## 4      5         0.6605        256.7557
## 5      6         0.4725        228.8524
## 6      7         0.9815        201.9790
```

4.2 Complexity-Penalized Risk for Adaptive Binning

```r
bins_20_sigma <- cut(x, breaks = 20, include.lowest = TRUE)
y_hat_20 <- tapply(y, bins_20_sigma, mean, na.rm = TRUE)
predicted_y_20 <- y_hat_20[as.character(bins_20_sigma)]
```

```r
resid_20 <- y - predicted_y_20
sigma_sq_hat_adaptive <- sum(resid_20^2, na.rm = TRUE) / (n - nlevels(bins_20_sigma))

risk_data <- data.frame(
  k = split_results$bins_k,
  rss = split_results$rss_after_split
)
risk_data$risk <- (risk_data$rss / n) + (2 * sigma_sq_hat_adaptive * risk_data$k * log(risk_data$k) / n

idx_adaptive <- which.min(risk_data$risk)
k_adaptive <- risk_data$k[idx_adaptive]
rss_adaptive <- risk_data$rss[idx_adaptive]

plot(
  risk_data$k, risk_data$risk,
  type = "b",
  pch = 19,
  col = "dodgerblue",
  xlab = "Number of Bins (k)",
  ylab = "Penalized Risk",
  main = "Complexity-Penalized Risk for Adaptive Regressogram"
)
points(k_adaptive, risk_data$risk[idx_adaptive], col = "red", pch = 19, cex = 1.3)
abline(v = k_adaptive, col = "red", lty = 2)
grid()
```




```r
cat("Optimal adaptive k:", k_adaptive, "\n")
```

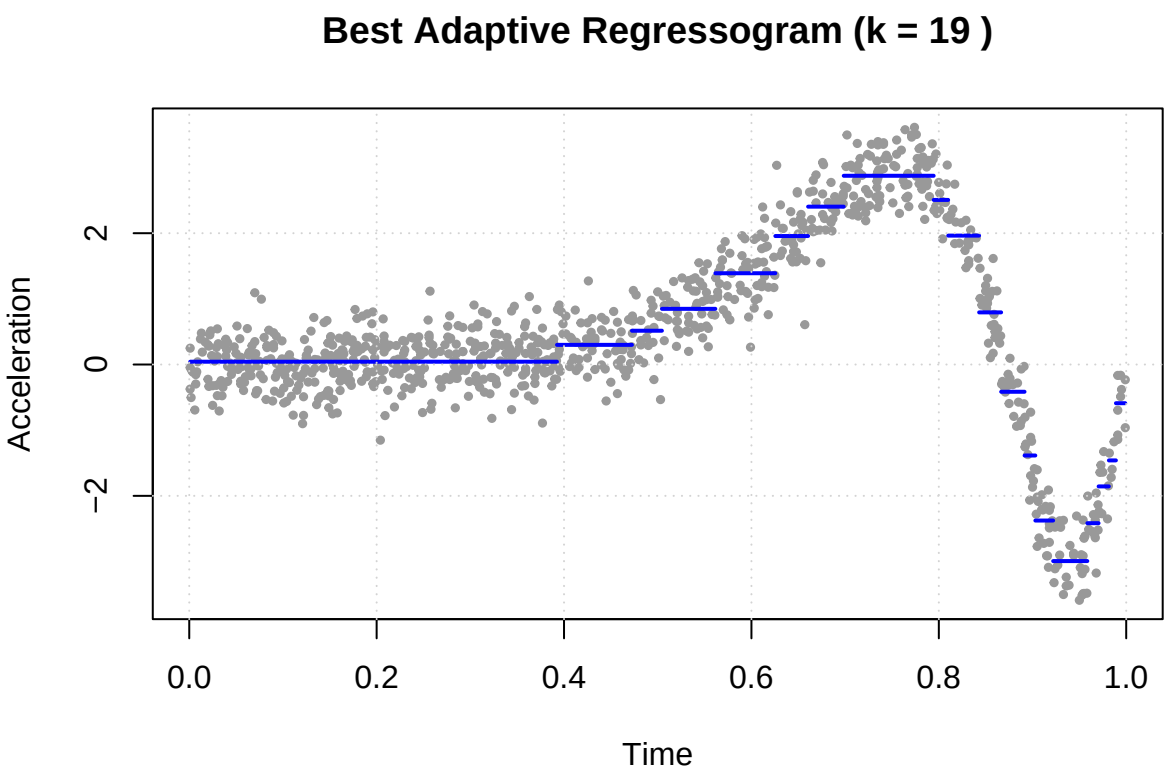
```
## Optimal adaptive k: 19
```

```r
cat("RSS at optimal adaptive k:", round(rss_adaptive, 4), "\n")
```

```
## RSS at optimal adaptive k: 129.0847
```

4.3 Best Adaptive Regressogram

```r
if (k_adaptive > 1 && k_adaptive <= nrow(split_results) + 1) {
  opt_splits <- sort(split_results$split_location[1:(k_adaptive - 1)])
  bin_bounds <- c(min(x, na.rm = TRUE) - 1e-9, opt_splits, max(x, na.rm = TRUE) + 1e-9)
  opt_bin_assign <- cut(x, breaks = bin_bounds, include.lowest = TRUE, right = FALSE)
  opt_bin_means <- tapply(y, opt_bin_assign, mean, na.rm = TRUE)

  plot(
    x, y,
    main = paste("Best Adaptive Regressogram (k =", k_adaptive, ")"),
    xlab = "Time",
    ylab = "Acceleration",
    pch = 19,
    cex = 0.5,
    col = "grey60"
  )

  for (i in seq_along(opt_bin_means)) {
    if (!is.na(opt_bin_means[i])) {
      segments(bin_bounds[i], opt_bin_means[i], bin_bounds[i + 1], opt_bin_means[i], col = "blue", lwd =
    }
  }
  grid()
}
```



The adaptive procedure selects `k = 19`, far smaller than the best fixed-bin regressogram. This does not mean the adaptive model is necessarily less expressive in practice. Rather, it uses fewer bins because the bin widths are allowed to adjust to the local geometry of the data.

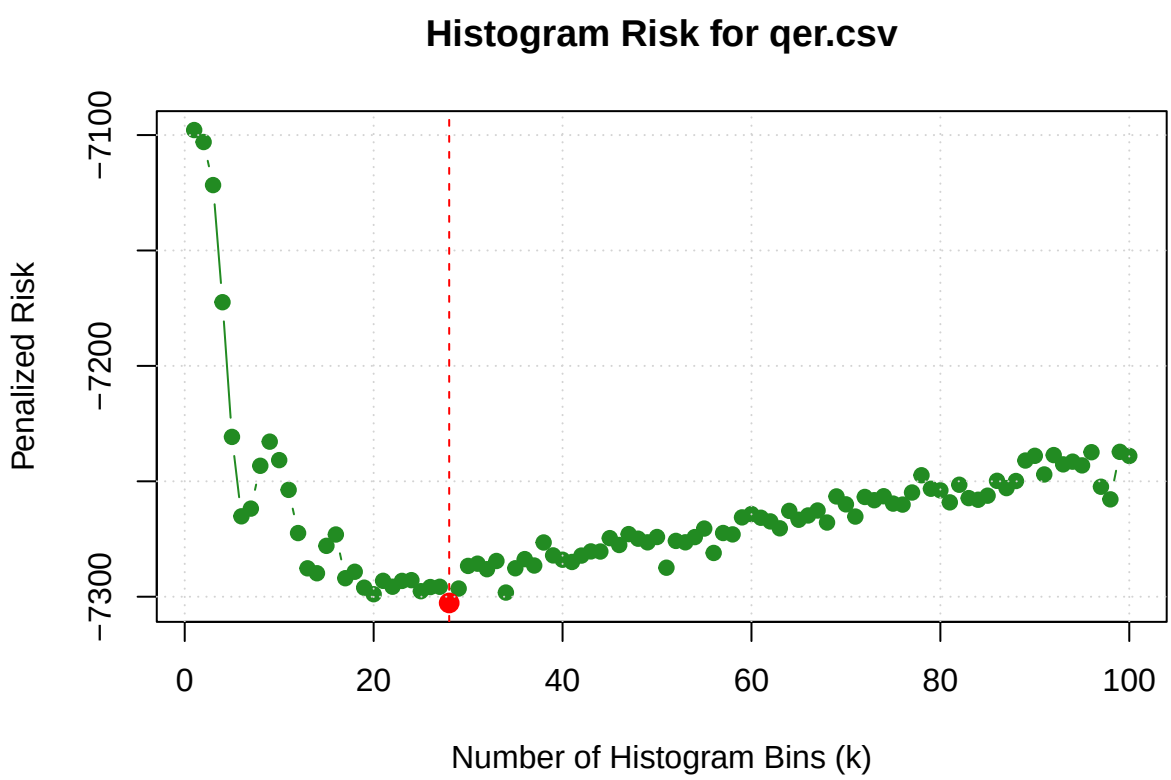
To make that comparison concrete, the optimal fixed-bin regressogram uses `k = 74` with RSS 123.1776, whereas the adaptive regressogram uses `k = 19` with RSS 129.0847. In other words, the adaptive model achieves nearly the same RSS with far fewer bins. In therms of the simple efficiency measure `RSS / k`, the adaptive fit retains about 0.95 of the residual efficiency per bin, indicating that adaptive partitioning allocates resolution where the signal varies most while using wider bins in smoother regions. This is the main efficiency gain of adaptation: comparable explanatory structure can be captured with far fewer bins when the partition is allowed to respond to the data rather than remain evenly spaced.

5. Histogram Risk Estimation on `qer.csv`

5.1 Penalized Histogram Selection

```
k_max_hist <- floor(min(100, n_qer / 5))
if (k_max_hist < 1) k_max_hist <- 1
k_hist_vals <- 1:k_max_hist

hist_risk <- data.frame(k = integer(), risk = numeric())
min_qer <- min(x_qer, na.rm = TRUE)
max_qer <- max(x_qer, na.rm = TRUE)

for (kh in k_hist_vals) {
  hist_breaks <- seq(min_qer, max_qer + 1e-9, length.out = kh + 1)
  hist_output <- hist(x_qer, breaks = hist_breaks, plot = FALSE, right = FALSE, include.lowest = TRUE)
  nj_counts <- hist_output$counts
```

```r
  NlogN <- -sum(nj_counts[nj_counts > 0] * log(nj_counts[nj_counts > 0]))
  NlogK <- -n_qer * log(kh)
  k_pen <- if (kh == 1) 0 else (kh - 1) + (log(kh))^2.5

  hist_risk <- rbind(hist_risk, data.frame(k = kh, risk = NlogN + NlogK + k_pen))
}

idx_hist <- which.min(hist_risk$risk)
k_hist_opt <- hist_risk$k[idx_hist]

plot(
  hist_risk$k, hist_risk$risk,
  type = "b",
  pch = 19,
  col = "forestgreen",
  xlab = "Number of Histogram Bins (k)",
  ylab = "Penalized Risk",
  main = "Histogram Risk for qer.csv"
)
points(k_hist_opt, hist_risk$risk[idx_hist], col = "red", pch = 19, cex = 1.3)
abline(v = k_hist_opt, col = "red", lty = 2)
grid()
```

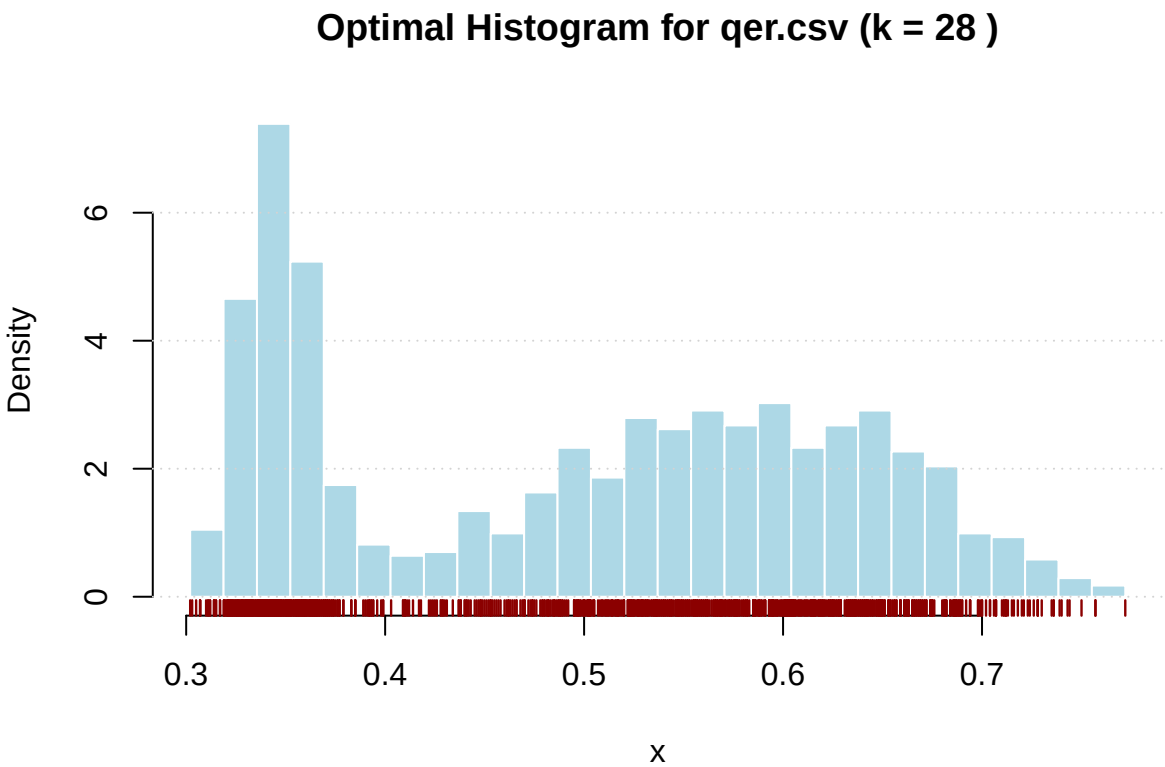



```r
cat("Optimal histogram k:", k_hist_opt, "\n")
```

```
## Optimal histogram k: 28
```

5.2 Optimal Histogram

```
hist_breaks_opt <- seq(min_qer, max_qer + 1e-9, length.out = k_hist_opt + 1)
hist(
  x_qer,
  breaks = hist_breaks_opt,
  freq = FALSE,
  right = FALSE,
  include.lowest = TRUE,
  main = paste("Optimal Histogram for qer.csv (k =", k_hist_opt, ")"),
  xlab = "x",
  ylab = "Density",
  col = "lightblue",
  border = "white"
)
rug(x_qer, col = "darkred", lwd = 1.2)
grid(nx = NA, ny = NULL)
```



6. Conclusion

This report develops a unified framework for **risk-based model selection in nonparametric smoothing**.

The main numerical results are:

- fixed-bin regressogram selected by covariance penalty: `k = 74`,
- fixed-bin regressogram selected by bootstrap optimism: `k = 74`,
- fixed-bin regressogram selected by 7-fold cross-validation: `k = 74`,

- LOWESS selected by 7-fold cross-validation: `f = 0.05`,
- adaptive regressogram selected by complexity-penalized risk: `k = 19`,
- histogram selected on `qer.csv`: `k = 28`.

Together, these results show that the same general principle applies across several different nonparametric estimators: These results illustrate how penalized-risk criteria balance model fit and complexity, providing a principled alternative to manual tuning.